

Communication Practices

We agreed on multiple communication channels. Each channel has its own strong and weak points and distinct area of applicability. Choosing the right channel can significantly improve the communication efficiency, just keep in mind that some messages may need to go through several channels at the same time. Most of these may look like common sense; The purpose of this document is **alignment**, not education.

Slack

Slack is a work-oriented instant messenger with simple voice call capability. Its main benefits are: it is fast, it is conversational and it is structured. Thus it is good for **quick questions**, **simple discussions** and **arranging things**.

Good practices:

- **Keep conversations out of DMs** (direct messages). You are not writing to a person - you are writing about a topic. You have two questions to the same person on two different topics? Might be worth asking them in two different channels. Don't have the right channel yet? Make it. Keeping conversations public boosts the team synergy.
- **Mute channels** you are not actively tracking. Slack allows you to receive the notifications for only the messages that explicitly tag you. Very useful for the channels you are only involved in occasionally - you can always say something or check what is going on there, without being distracted by the ongoing conversations unless they explicitly ask for you.
- **Tag people** you want to react to your message. You don't have to tag every message, but generally expect people to have the notifications off and skim through the messages. If you ask somebody a question - tag them. If you want somebody else to be aware of the thread conversation you just had - drop a short "fyi, @A, @B, @C", and they will know you are expecting them to actually read the thread.
- **Expect async** by default. We are working on different schedules in different timezones. Sometimes people are just too busy to reply right away. Defaulting to asynchronous messaging allows the conversation flow without distracting people from what they are doing. If you need a fast reaction to your message - be upfront about it so that it is clear from the push notification popup.
- **Use huddle as needed** (in-Slack voice chat) to speed up the conversations that have a lot to talk through. But remember, it has no transcript, so whatever answers or decisions are made there - write a brief follow-up in the chat so that the teammates who missed the huddle will still be able to catch up.
- **Extract decisions and findings**. Slack channel is an ever growing thread. An important decision made two weeks ago will be close to impossible to find. Slack is good for

discussions, but the *results* of discussions should be documented somewhere easy to find - an email for a paper trail or a documentation for referencing.

- **Keep messages short.** If you need to write a screen of text in Slack - most likely you don't need to write it in Slack. Write a document instead, and send a link to Slack - it will be much easier to review, comment and update it.
- **Save for later** and **snooze** messages as needed. These two great features of Slack make it easier to keep track of what you need to react to, but can't right now. "Save for later" saves the message into your "Later" tab so it is easy to find later - as long as you keep your "Later" tidy by removing the processed messages. "Snooze" just repeats the push notification and marks the message unread some time later. Useful when someone tags you, you read the message but decide to respond later. By snoozing it you don't need to remember about it - Slack will remind you about it.

Google Meet

Realtime voice / video call with the recording and AI transcription allows the **highest information density** of the communication if it stays focused and engaged. An hour meeting can save a week of emailing back and forth. On the other hand, it is one of the most disruptive communication channels and can easily become a time waste if it is not kept focused. Thus it is good for dense **Q&A**, **presentations**, **alignment** and **brainstorming** / exchanging ideas.

Good practices:

- Ensure the meeting is **transcribed**, and **recorded** if needed.
- **Prepare agenda** beforehand. It doesn't mean the meeting shouldn't side-track. But if there is no agenda - there shouldn't be a meeting at all. Realtime group video calls are one of the most time-consuming ways of communication and if people don't know the purpose of the meeting - it can easily turn into a waste of time.
- **Follow agenda** at the meeting. While some off-topics might be useful and helpful, generally they should not prevent us from achieving the goal of the meeting. Some of the off-topics surfacing at the meeting may benefit from being recorded into the agenda of the separate discussion rather than being discussed on the spot.
- **Respect the timing.** People might not be able to stay beyond the allocated time slot. Aim to achieve the meeting goal within the arranged time. It is ok to conclude early. It is almost never ok to drag it longer. If the conclusion is not achievable in the available time - ask people if they can stay longer, but expect to schedule the follow-up meeting instead.
- **Document decisions and findings.** Transcription is helpful. Looking through transcripts of three different hour-long meetings to find that one decision - is not. Any decisions, conclusions or findings produced in the meeting should be documented in the

appropriate channel. If it is a long-lasting decision - that's for the persistent documentation (and maybe an email). If it is an action item - that's a Jira task. If it is a conclusion of a short-term value, e.g. a status update - it could be a Slack message in the appropriate channel. Just don't leave it sitting in the meeting recording alone.

- **Keep invites focused.** Only invite people who *matter* for the conversation. An uninterested party sitting out a meeting is a waste of time. If you want to let someone know the meeting is happening, but don't expect them to actively contribute to it - invite them as *optional*, so that they can decide if it is valuable enough for them to participate.

Persistent documentation

The key strong points of the persistence documentation are: long-lasting, structured, collaboratively editable. A good documentation is referenceable, searchable, version controlled and access controlled. Thus the two main good use cases for the persistent documentations are hand-off (including to the future-you) and collaboration.

Generic good practices

- **Keep documents small and focused.** 20 x 5-page documents with clear name, target audience and purpose are generally much easier to manage and read than a single 100-page document containing everything.
- **Low coupling, high cohesion** principle. Things that have a high chance to change together - should live together. Things that have a low chance of changing together - should live in separate documents. For example, a list of procedural steps of the training module and AI agent architecture have no reason to be in the same doc. A list of procedural steps and a list of assets and artefacts needed to render those steps might benefit from being documented together - a change to the list of steps has a good chance to end up affecting the list of the assets as well.
- **Separate current state and change logs.** People generally read persistent documents for two main reasons - either to find out the *current* state. Or to figure out *why* the current state is different from their *expectations*. In the first case they are interested in what is relevant right now, and any historical data is a distraction. In the second case they are interested in tracking the *changes* and *reasoning* behind the changes. If both current state and historical reasoning are interleaved in the same document - it makes the document inconvenient to read for everyone. Separate change logs into the separate sections or even different documents where appropriate.
- **Assign gatekeepers.** While collaborative editability is a huge convenience, many people editing the same document at the same time end up in a chaos. Gatekeeper is a person nominated to moderate the document editing. They don't have to author the document directly, but they are responsible to review and moderate the edit suggestions made by other people to ensure the document stays focused and well-structured. It

might be convenient to declare the gatekeeper in the page header / footer for the internal documents.

- **Declare privacy and confidentiality level.** Unintentional disclosure of confidential information can hurt. Explicit "confidential" markers on a document makes a person think before sharing its content. Explicit "public" markers on a document makes a person think before writing any confidential information into it.
- **Keep documents up to date.** Implicitly outdated documentation can be more dangerous than no documentation at all. Mark obsolete documents so that the reader knows it is unreliable. Update the outdated document when possible and reasonable. Documentation maintenance cost is generally lower than the cost of somebody confidently acting on the basis of the irrelevant document.
- **Recognize the document lifetime.** Different documents have different lifetimes. A collaboratively compiled meeting agenda is only relevant till the end of the meeting. A to-do item is only relevant till it is done. An architecture convention is relevant for as long as the product is alive.
- **Keep in mind the target audience.** The same document written for the internal technical team vs for the public pitch in front of the potential investors ends up being two completely different documents. Writing the document, keep in mind who you are writing it for, what is their language and what is their common sense.
- **Add a short intro** at the beginning of the document. A short paragraph at the top of the document describing what it is and who it is for (e.g. "high-level technical architecture overview for IRB") makes orienteering in the documents much easier and faster.
- **Embed enough context, cross-reference documents**, but keep document life-time in mind. Extension of target audience principle, keep in mind that the reader might not remember all the details you remember when you write the document. As such, reminding the key concepts and guiding to the related documentation can help a lot. Generally, it is ok to reference long-lived documents from a short-lived document. But not the other way - short-lived documents getting outdated would trigger frequent edits in the long-lived document for no good reason.
- **Format and structure.** Structure the document so that it is easy to find the relevant piece of information without reading every single line of text. Format the document for skimming - use bold, italic, headers, titles, colours and highlights so that it is easy to scan the document and pick the relevant bits while skipping the irrelevant..
- **Inform interested parties about substantial changes.** If a substantial change or a new document has landed - take a moment to write an email or a message in a relevant Slack channel to let the teammates know of it. No one starts their day by re-reading all the documents they already read yesterday, so don't expect people to notice the change unless you notify them.

OneDrive / Docs good practices

- **Don't share publicly**, unless the document is public. Any non-public document should not be shared by "visible by anyone with the link". If you need to share the file to an unknown third party - declare the document as public or make a public copy of it with revised content.
- **Keep version control**. Documents that can go stale (e.g. exported as a file) should include a version marker (timestamp can be a version marker). Documents that can be discussed comparatively against several versions of it should also include a version marker. Google Workspaces allows authors to create a **named version** in the document history. It is important so that people can confirm they are talking about the same (or different) version of the document.
- **Use comments and suggestions** if you are not a gatekeeper of the document.

Code documentation good practices

- **Treat code as documentation**. Write it so that another human, including future you, would be able to understand your intention. Code is the best implementation documentation, because it is always in sync with the implementation.
- **Keep docs next to the code**. If something is only useful when editing this specific code file - write it in this code file. If something is only useful when using this class or than method - add it to the corresponding documentation comment. Only separate the documentation that should be shared without the code (e.g. API conventions) or describe high-level concepts that do not relate to a specific piece of code (e.g. architecture conventions).
- **Keep the documentation up to date**. Keeping docs in the same repo as the code and treating documentation update a part of merge request allows to keep the docs relevant across branches.
- **Keep version control**. If you store docs in git repo - it does the job for you. If you don't - make sure whatever you use keeps track of who, what, when.
- **Version protocol definitions**. Any protocol definition that can have more than one simultaneous implementation should be explicitly versioned. Network data transfer protocols, persistent data storage protocols, etc should always include the protocol version in the message so that the receiving endpoint could validate its expectations.

Jira

Jira task *is* a communication tool. It eases keeping track of the pending action items and provides the status updates without even need for people to talk. It also communicates the overall status of the project, workload distribution, bottlenecks and much more. Tasks are living entities should be used to **streamline the workflow** and **communicate action items**, not for historical archives.

Good practices:

- **Create tasks** on demand. If you ask somebody to do something non-trivial - something they cannot just do right away - create a task and assign it to them. Don't expect people to remember everything they were asked to do in every slack channel and every email thread.
- **Use a descriptive summary.** The purpose of the summary is not to explain what it is, but to be recognizable at a glance - it is a trigger, not an instruction. Use description to explain what the task is about in detail.
- **Write proper description.** Imagine you are going to ask somebody to do something for you on Slack or email - put the same thing into the task description instead. Add a bit of context on top where appropriate - drop a link to the relevant documentation or a Slack thread, etc. The easier you make it for the assignee to understand what you are asking them to do - the faster the task will get done.
- **Queues and stations** are also means of communication. Queues are statuses representing work items waiting to be worked on. If you ask someone to do something for you - drop it into their queue. Stations are the statuses representing work items actively worked on. When you start doing a task - pull it into your station (in-progress) to let the requester know it is being worked on. When you finish the task - push it into the downstream queue. If you stop working on a task midway - push it back into the queue to let the requester know it is not being worked on anymore.
- **Assign tasks.** Assignee is not a person "responsible for a task", but a person who is expected to *work* on it. If you ask somebody to do something - *assign* it to them, otherwise they wouldn't know it is for them. If someone assigned you a task you cannot do - communicate it and find who to reassign to.
- **Use filters and dashboards.** A good dashboard allows you to see your work items at a glance. A good filter allows you to look at the slices of the board that are the most relevant for you. The most common must-have filter - "everything assigned to me" and "everything in my value-stream".
- **Keep your tasks status up-to-date.** Don't forget to move tasks between columns as you start and stop working on it. Keep Jira open always during your workday, just like Slack or email.

Gatekeepers: Ivan D , ffronchetti@lsu.edu

- **Tasks can produce tasks.** If you start working on something and realise you need someone else to do something for you to continue - create a new task, assign it to them and set it as a **blocker** to your tasks.
- **Flag blocked tasks as impediments** when they are actively blocked. Jira has clear visualization for it that allows everyone to see which tasks cannot be worked on.
- **Return tasks for approval.** Don't just silently close tasks done - drop them into the review / approval queue of the original requester so that they know to check at and close or return for rework.